Hospital Management System C++ Project Proposal Using Object Oriented Programming

Submitted By: **Habib ur Rehman**
Roll Number: BSCS25085
Course: Object Oriented Programming (OOP)
Semester: Spring 2026

Abstract

This project is a Hospital Management System developed in C++ using Object Oriented Programming concepts. The purpose of this project is to automate hospital operations including patient management, doctor management, appointment booking, billing, prescriptions, authentication, and report generation. The system demonstrates major OOP concepts such as inheritance, polymorphism, encapsulation, abstraction, operator overloading, exception handling, modular programming, and file handling.

Introduction

Hospitals require an organized and efficient system to manage patient records, doctors, appointments, billing, and prescriptions. Traditional manual systems are slow and error-prone. This project provides a console-based computerized solution using C++ to improve efficiency, accuracy, and data organization.

Problem Statement

Manual hospital management systems create several problems such as data loss, appointment conflicts, billing mistakes, inefficient record management, and difficulty in retrieving information. The goal of this project is to build a digital Hospital Management System capable of managing hospital operations effectively through proper software engineering and Object Oriented Programming techniques.

Objectives

• Implement Object Oriented Programming concepts practically.
• Demonstrate inheritance, polymorphism, abstraction, and encapsulation.
• Create a modular project using multiple .cpp and .h files.
• Implement secure authentication for Admin, Doctor, and Patient.
• Manage appointments, billing, prescriptions, and reports.
• Use file handling to permanently store and retrieve records.
• Improve efficiency and organization in hospital management.

System Modules

1. Authentication Module – Login system for Admin, Doctor, and Patient.
2. Patient Management – Add, update, search, and remove patient records.
3. Doctor Management – Manage doctor details and availability.
4. Appointment Module – Book, cancel, and manage appointments.
5. Billing Module – Generate hospital bills and payment records.
6. Prescription Module – Store medicines and treatment notes.
7. Reports Module – Generate daily and monthly hospital reports.
8. File Handling Module – Save and retrieve information from text files.

Inheritance Structure

The project follows the inheritance hierarchy:

Person → Patient, Doctor, Admin

The Person class acts as the base class containing common attributes such as ID, name, password, age, and contact information. Patient, Doctor, and Admin classes inherit these features and extend them according to their specific functionality.

Polymorphism

Runtime polymorphism will be implemented using virtual functions.

Examples:
• virtual void displayMenu();
• virtual void generateReport();

Different derived classes will override these functions according to their required behavior.

Encapsulation and Abstraction

All data members will remain private and will be accessed through public getter and setter functions. Abstract classes and pure virtual functions will hide unnecessary implementation details and expose only essential functionality to users.

Operator Overloading

Operator overloading will improve code readability and simplify operations.

Examples:
• += for adding balance
• -= for bill deduction
• == for comparing objects
• << for displaying object information

Exception Handling

Custom exceptions will be implemented to handle invalid inputs, incorrect login attempts, appointment conflicts, insufficient balance, and file handling errors.

Project File Structure

Source Files (.cpp)

- main.cpp → Program execution starts here.
- Person.cpp → Defines common person-related functions.
- Patient.cpp → Patient management implementation.
- Doctor.cpp → Doctor management implementation.
- Admin.cpp → Admin management implementation.
- Appointment.cpp → Appointment handling implementation.
- Bill.cpp → Billing system implementation.
- Prescription.cpp → Prescription system implementation.
- FileManager.cpp → File handling implementation.
- Validator.cpp → Input validation implementation.
- Menu.cpp → Console menu implementation.
- Utility.cpp → Helper functions implementation.

Header Files (.h)

- Person.h → Base class declarations.
- Patient.h → Patient class declarations.
- Doctor.h → Doctor class declarations.
- Admin.h → Admin class declarations.
- Appointment.h → Appointment class declarations.
- Bill.h → Billing class declarations.
- Prescription.h → Prescription class declarations.
- FileManager.h → File handling function declarations.
- Validator.h → Validation function declarations.
- Menu.h → Menu function declarations.
- Utility.h → Helper function declarations.

TXT Files Used for Permanent Data Storage

The system will permanently store and retrieve data using the following text files:

- patients.txt → Stores patient records.
- doctors.txt → Stores doctor information.
- admin.txt → Stores admin login credentials.
- appointments.txt → Stores appointment details.
- bills.txt → Stores billing records.

• prescriptions.txt → Stores medicines and prescriptions.
• reports.txt → Stores generated hospital reports.
• security_log.txt → Stores login attempts and security activities.

How Data Will Be Stored and Retrieved

File handling in C++ will be implemented using **fstream**. " "Whenever a user adds a patient, books an appointment, generates a bill, or creates a prescription, the system will write that information into the corresponding .txt file using file streams.

Example:
• When a new patient is registered, the patient's ID, name, age, gender, and disease information will be stored in patients.txt.
• When the program starts again, the system will read existing records from the text files and load them into objects for further processing.

Functions such as **ofstream** will be used for storing data, while **ifstream** will be used for retrieving data. " "This ensures that records remain permanently saved even after the program closes.

Estimated Project Size

• 12–15 Header Files
• 12–15 CPP Files
• 1 Main File
• Approximately 2500–3500 lines of code

Technologies Used

• C++ Programming Language
• Object Oriented Programming
• File Handling
• Multi-File Programming
• Console-Based Interface

Expected Output

• Patient registration and management
• Doctor management system
• Appointment booking and cancellation
• Billing and payment management
• Prescription management
• File-based permanent data storage
• Report generation

Advantages

• Organized hospital management
• Reduced manual work
• Faster access to records
• Better appointment scheduling
• Strong implementation of OOP concepts
• Practical software development experience

Conclusion

The Hospital Management System is a medium-scale OOP project designed to automate hospital operations efficiently. The project demonstrates major Object Oriented Programming concepts including inheritance, polymorphism, abstraction, encapsulation, operator overloading, exception handling, and file handling. This project will provide practical experience of real-world software development using C++ and improve understanding of modular programming and permanent data management.